Exact negacyclic polynomial multiplication on the GPU: the optimizations behind `fherma-polymul`

Working notes

14 October 2026

Abstract

These notes document the optimization work on our entry to the FHERMA challenge *Polynomial Multiplication (GPU-Accelerated)*, which asks for the exact product $c = a \cdot b \bmod (X^N + 1, q)$ of two polynomials with coefficients of up to a few thousand bits. The solution uses a residue number system (RNS), a negacyclic number-theoretic transform (NTT) per residue prime, and a Chinese remainder (CRT) reconstruction. For each optimization we give the problem, how it was diagnosed, what was changed and what it measured, including the changes that did not help. At the benchmark point $N = 32768$, $W = 868$ the time per multiplication on our development machine went from 3.07 ms to 700 µs; the code was then generalized to the point $N = 65536$, $W = 1747$ that now decides the ranking, where it first took 3.09 ms; kernel fixes brought it to 2.6 ms, and reading and writing the caller's memory directly through HMM to about 2.3 ms (Point #2: about 610 µs).

Contents

1 The problem and how it is scored **2**

2 Algorithm **3**
2.1 Residue number system . . . 3
2.2 Pipeline . . . 3
2.3 Modular arithmetic . . . 3
2.4 GPU schedule . . . 4

3 How we measured **4**

4 Optimizations **4**
4.1 CRT: from one thread per coefficient to prime groups . . . 5
4.2 Host side: a copy pool instead of page registration . . . 5
4.3 Chunking: fewer, shaped chunks . . . 6
4.4 Zero-copy decompose and CRT . . . 6
4.5 Worker wake-up: spinning between calls . . . 6
4.6 Zero-filling the output on a helper thread . . . 7
4.7 Overlapping a's forward NTT with b's transfer . . . 7
4.8 Decompose: register blocking . . . 7
4.9 NTT twiddles: one load instead of two . . . 8

5 Generalizing to Point #1 **8**

6 Optimizations for Point #1 **8**
6.1 Twiddles generated from two small tables . . . 9
6.2 CRT split over limb ranges . . . 9
6.3 Host overhead at Point #1 . . . 10
6.4 A container-only slowdown . . . 10

7 Getting closer to the transfer floor **10**
7.1 Reading and writing the caller's memory through HMM . . . 11
7.2 Chunking in HMM mode . . . 11
7.3 Tried and not adopted . . . 11
7.4 The zero-fill inside the challenge image . . . 12
7.5 Further attempts . . . 12

8 Where the time goes now **12**
8.1 Point #2 . . . 12
8.2 Point #1 . . . 13

9 What did not help **13**

10 Summary at Point #2 **14**

A Runtime settings **14**

B Reproducing the measurements **14**

1 The problem and how it is scored

Specification. The kernel `polynomial-multiplication/negacyclic@1.0.0` computes, for two polynomials a, b of degree $< N$ with coefficients in $[0, q)$,

$$c \;=\; a \cdot b \bmod (X^N + 1,\; q),$$

exactly, with every output coefficient canonical in $[0, q)$. N is a power of two; q is the largest prime below 2^W with $q \equiv 1 \pmod{2N}$. A coefficient is stored as $L = \lceil W/32 \rceil$ little-endian 32-bit limbs, so an operand is an $N \times L$ array of `u32`. The specification sets no upper bound on N or W and lists five points: $(N, W) = (64, 32)$, $(1024, 512)$, $(8192, 256)$, $(32768, 868)$ and $(65536, 1747)$.

Scoring. The score is the median wall-clock time of one call of `fherma_run`, measured from the moment a and b are available in host memory to the moment c is complete in host memory. *Copies to and from the GPU are therefore inside the timed window.* Setup (`fherma_init`) is not timed but may depend only on N, W, L and q. Results must be exact; FHERMA re-tests on 20 random seeds per point and on non-public inputs.

Benchmark points. Since 28 September 2026 the ranking is decided at **Point #1**, $N = 65536$, $W = 1747$, $L = 55$; the original point, $N = 32768$, $W = 868$, $L = 28$, is now **Point #2** and is shown beside it. An entry with no Point #1 result ranks behind every entry that has one.

Hardware. All measurements in these notes come from our development machine, a shared QEMU virtual machine with 96 virtual CPUs (x86-64-v2, no AVX) and two NVIDIA L40S cards (`sm_89`, 46 GB, PCIe Gen4 x16). The ranked runs use an NVIDIA RTX PRO 6000 (`fherma-gpu-rtx6000`), a different architecture with a faster PCIe link, so absolute times are not comparable; the relative effects of host-side changes in particular may differ there.

Data volume. At Point #2 an operand is $32768 \times 28 \times 4$ bytes $= 3.67$ MB, so each call moves 11 MB across PCIe (two operands in, one out); at Point #1 an operand is 14.4 MB and a call moves 43 MB. On our Gen4 link, pinned DMA runs at about $24.5\,\mathrm{GB\,s^{-1}}$, which puts a floor of roughly 450 µs under Point #2 and 1.75 ms under Point #1 before any compute.

2 Algorithm

2.1 Residue number system

Choose K primes $p_j < 2^{31}$ with $p_j \equiv 1 \pmod{2N}$ (so that each has a primitive $2N$-th root of unity ψ_j), and let $P = \prod_j p_j$. The integer (not reduced) negacyclic product $\tilde{c} = \sum_{i+k=n} a_i b_k - \sum_{i+k=n+N} a_i b_k$ satisfies $|\tilde{c}_n| < Nq^2$. If $P > 2^{20} \cdot 2Nq^2$, then $\tilde{c}$ is determined by its residues modulo the p_j, with a margin of 20 bits that makes the floating-point rounding below exact. The number of primes needed is

$$K \approx \frac{2W + \log_2 N + 1 + 20}{31},$$

which gives $K = 58$ at Point #2 and $K = 115$ at Point #1. The primes are taken from just below 2^{31} downwards in steps of $\max(2N, 2^{16})$; all of them lie above $2^{30.8}$.

2.2 Pipeline

1. **Decompose.** Each coefficient's L limbs become its residue modulo each prime: $x \bmod p = \sum_l x_l \cdot (2^{32l} \bmod p) \bmod p$, with the constants $2^{32l} \bmod p$ precomputed (with Shoup companions, §2.3). Output: two $K \times N$ arrays of residues.

2. **Forward NTT** per prime, Cooley–Tukey, with twiddles $\psi^{\mathrm{bitrev}(i)}$ so that the negacyclic twist is folded into the transform and no pre- or post-multiplication by powers of ψ is needed.

3. **Pointwise product** in the NTT domain, multiplied by the constant $N^{-1} \cdot (P/p_j)^{-1}$, so that the inverse transform directly yields $y_j = \tilde{c} \cdot (P/p_j)^{-1} \bmod p_j$.

4. **Inverse NTT** per prime, Gentleman–Sande, natural order out.

5. **CRT.** With $v = \mathrm{round}\left(\sum_j y_j/p_j\right)$, computed in double precision, the centred value is $\tilde{c} = \sum_j y_j(P/p_j) - vP$, and modulo q

$$c \;=\; \sum_j y_j \cdot M_j \;+\; (-vP \bmod q) \pmod{q}, \qquad M_j = (P/p_j) \bmod q,$$

   where both M_j and the $K+1$ possible values of $-vP \bmod q$ are precomputed. The final reduction modulo q uses a quotient estimated in double precision from the top limbs, followed by at most two subtractions.

2.3 Modular arithmetic

All residue arithmetic is on 32-bit words with $p < 2^{31}$, so a lazily reduced value below $2p$ still fits in a word.

- **Shoup multiplication** by a constant w: with the precomputed companion $w' = \lfloor w \cdot 2^{32}/p \rfloor$, $r = aw - \lfloor aw'/2^{32} \rfloor \cdot p \pmod{2^{32}}$ lies in $[0, 2p)$ for any 32-bit a; one conditional subtraction finishes it. Three integer multiplies, no 64-bit product.

- **Montgomery multiplication** $\mathrm{mont}(a, b) = ab \cdot 2^{-32} \bmod p$ for the pointwise product of two variables.

2.4 GPU schedule

- **Four-step NTT split.** The $\log_2 N$ stages are split at row length $S = 2^{\lceil \log_2 N/2 \rceil}$: stages whose butterflies span a distance $\geq S$ pair elements of the same column (stride S), the others pair elements within a contiguous row. Each pass keeps a tile of $T = 2048$ residues in shared memory for all its stages. At $N = 32768$: $S = 256$, $R = N/S = 128$, and a column-pass block holds $C = T/R = 16$ columns.
- **Kernels per call.** `k_decompose` (per input chunk), `k_ntt_cols` and `k_ntt_rows_fwd` on a, `k_ntt_cols` on b, `k_ntt_rows_mul` (forward rows of b, pointwise product, inverse rows), `k_ntt_cols` (inverse), and `k_crt` (per output chunk).
- **Streams.** Host-to-device copies run on one stream, kernels on a second, device-to-host copies on a third, chained by events, so that copying chunk $i + 1$ overlaps decomposing chunk i, and copying output chunk i overlaps the CRT of chunk $i + 1$.
- **Shared schedule.** Tile geometry, index maps and per-phase steps live in one header used by both the GPU kernels and a CPU emulation, which runs the same schedule block by block and phase by phase and checks that no NTT stage touches an element twice. The CPU build therefore tests the GPU's indexing, not just its arithmetic.

3 How we measured

Correctness. Every change was checked with `run_tests.sh`, which generates test points with Python oracles (an exact $O(N^2)$ reference for small N, a GMP Kronecker-substitution reference for large N) and verifies the outputs bit for bit. The matrix now has 21 points: every NTT geometry, every limb edge ($L = 1$, q filling its limbs, q one bit into a new limb, $L = 64$), the five points of the specification, $N = 2^{17}$, and an all-$(q - 1)$ worst case in case 0 of each large point. Both the CPU build and the GPU build run it. NVIDIA's `compute-sanitizer` (memcheck, racecheck, synccheck) was run after every kernel change.

Fuzzing. `fuzz.py` adds 61 more points (every 32-bit width boundary up to 2048, N from 2 to 2^{22}, 30 random sizes) with six adversarial input patterns each (all $q - 1$, random, zero, the $x^{N-1} \cdot x^{N-1}$ wrap, the identity, alternating), under every copy mode and twiddle path and on the CPU build, against a GMP reference that is itself checked against the schoolbook product for small N: no failures. Out-of-range sizes are refused with a message.

Performance. The development server is shared, and other users' jobs moved wall-clock medians by 50 µs or more, with outliers up to 1.3 ms. Three practices made the measurements usable:

1. **Interleaved A/B runs**: alternate the builds or settings being compared over 5–20 rounds of 21 cases, and compare medians and lower quartiles of the per-round medians.
2. **Kernel times and the GPU span from Nsight Systems**: the duration of each kernel and the span from a call's first transfer to its last are almost unaffected by host noise.
3. **Elimination**: build scratch variants that skip one part of a kernel (the stores, the arithmetic, the table loads) and attribute the time by difference. Combined with instruction counts from the SASS (`cuobjdump`), this found the real bottleneck every time our first guess was wrong.

4 Optimizations

Unless stated otherwise, numbers are for Point #2. Each subsection gives the problem, the diagnosis, the change and the measured effect.

4.1 CRT: from one thread per coefficient to prime groups

Problem. The CRT kernel took 47 µs in one launch over all 32768 coefficients, but about 38 µs per launch *regardless of launch size* when the output was split into chunks, so the default 8 chunks cost 308 µs. One coefficient was handled by one thread, in blocks of 64 threads: a chunk of 4096 coefficients was 64 blocks of two warps on a 142-SM GPU.

Diagnosis. Removing each part of the kernel in turn gave, per chunk:

part of `k_crt`	cost per chunk
products (58×28 multiply-accumulates)	13.5 µs
loading Y, storing c, launch	12 µs
rounding estimate $\sum_j y_j/p_j$	2.3 µs
final reduction modulo q	2 µs

No single part was slow: every phase was waiting on latency, because there was far too little parallel work per launch.

Change. (1) Each block first stages its tile of Y (K residues for each of its coefficients) into shared memory in one coalesced pass, with all loads in flight together. (2) The K primes are split into 4 groups, and each coefficient gets 4 threads, each computing a partial sum over its group, carried down to $L+2$ words. A last phase adds the 4 partials and $-vP \bmod q$ and reduces modulo q. Blocks are 32 coefficients $\times$ 4 groups = 128 threads; a warp is one prime group and 32 consecutive coefficients, so its loads of the table M are uniform.

Result. 308 µs $\to$ 92 µs at 8 chunks; 47 µs $\to$ 38 µs in one launch. The staging step alone (without the groups) gave only 308 µs $\to$ 232 µs: latency of global loads was part of the problem, the lack of parallelism the larger part. End to end this was worth only 1–2 % at the time, because the host side dominated (next section).

4.2 Host side: a copy pool instead of page registration

Problem. With the GPU work at about 1.1 ms of a 2.1 ms call, roughly half the call was spent on the host. The inputs arrive in freshly allocated `std::vector`s, and the output must be returned as a new vector.

Diagnosis. A host-only benchmark measured each step for one 3.67 MB operand on this VM:

step	median
`cudaHostRegister` + unregister of the caller's buffer	400 µs
`memcpy` into a pinned buffer, 1 thread	300 µs
`memcpy` into a pinned buffer, 16 threads (cache-hot)	13 µs
zero-filling a new output vector (`resize`)	237 µs
pinned DMA, either direction	150 µs
pageable `cudaMemcpy`, either direction	357 µs

Registering the caller's pages, the fastest mode before, was the most expensive way to reach the GPU; one-thread copies through pinned memory were next.

Change. A pool of `std::thread` workers copies each input chunk into a pinned staging buffer, then the chunk's DMA and decompose are enqueued. The copy is cut into 64 kB pieces that the caller and the workers claim from a shared counter. The counter packs a job number with the piece index and pieces are claimed by compare-and-swap, so a late worker can never claim a piece of a newer job with an older job's parameters, and the caller never waits for a worker that is not awake. The output is copied out of a pinned buffer by the same pool, chunk by chunk, as each chunk lands. A first version used static slices per worker; it had a shutdown hang (spinning workers never checked the quit flag), found by an experiment and fixed.

Result. 2104 µs → about 880 µs per call (best setting before → new default), a factor of 2.4.

4.3 Chunking: fewer, shaped chunks

Observation. After the copy pool, more chunks made things *worse*: 8 input chunks cost about 250 µs more than 2. We first attributed this to about 25 µs of CUDA API calls per chunk. That explanation was wrong: with zero-copy output (§4.4), where a chunk is only one kernel launch and one event, extra chunks still hurt. The cost is a fixed overhead per parallel copy in the pool.

Changes and results.

- Default pipeline depth reduced from 8 chunks to 2.
- Input and output chunking were decoupled, and the chunks of each operand can be given relative weights (settings in Appendix A). An output split of 1:2:1 lets the first output DMA start after a smaller first CRT chunk and leaves less to copy after the last DMA. Over 10 interleaved rounds: 715 µs → 700 µs median, lower quartile 710 µs → 695 µs. Now the default.
- Uneven *input* chunks (a small first chunk of a, a small last chunk of b) did not help; a 3:1 or 7:1 split of b made the GPU span worse (625 µs and 647 µs against 591 µs–610 µs), because the larger first chunk delays b's first transfer.

4.4 Zero-copy decompose and CRT

Idea. Map the pinned staging buffers into the device address space so that `k_decompose` reads its limbs directly over PCIe and `k_crt` writes its limbs directly to host memory, removing the separate DMA, event and stream hop of every chunk.

Result. Over 5 interleaved rounds, mapped input saved about 15 µs–20 µs (2 %), well inside the 790 µs–940 µs spread between rounds. Kept as an opt-in mode (`FHERMA_IN/OUT=mapped`); the default stays staged. It may behave differently on the reference machine.

4.5 Worker wake-up: spinning between calls

Problem. The copy workers sleep on a condition variable between calls. Timing each input copy showed the first copy of a call taking 240 µs–270 µs instead of 60 µs–110 µs: waking sleeping threads in this VM costs about 150 µs–180 µs. The harness leaves about 13 ms between calls (it writes the previous answer and reads the next inputs from files).

Change. `FHERMA_SPIN_MS` (default 20 ms): after a call, the workers keep spinning for that long before sleeping. A test without the `pause` instruction in the spin loop (to rule out the hypervisor descheduling spinning virtual CPUs) made no difference.

Result. Over 5 interleaved rounds: 860 µs → 760 µs (staged) and 840 µs → 763 µs (mapped); every spinning round beat every sleeping round. The cost is 15 threads busy-waiting through the gap between calls; whether the grader allows that is an open question.

4.6 Zero-filling the output on a helper thread

Problem. The output vector must be sized (`resize`, a zero fill of about 350 µs here) before chunks can be copied into it. That happened after all GPU work was enqueued, about 300 µs into the call, so it ended near 660 µs, after the first output chunk had landed (about 620 µs): it was on the critical path.

Change. A dedicated helper thread (spinning and sleeping like the pool) zero-fills the output from the start of the call, while the calling thread copies the inputs. A guard object waits for it on every exit path, since it writes into the vector being returned.

Result. 736 µs → 708 µs over 6 interleaved rounds (5 consistent, one outlier).

4.7 Overlapping a's forward NTT with b's transfer

Problem. The forward transforms of a and b ran together after b had fully arrived, but a's does not need b.

Change. The forward transform of a (column pass and a new row-only kernel `k_ntt_rows_fwd`) is enqueued right after a's decompose, so it runs while b is still crossing PCIe; `k_ntt_rows_mul` gained a mode that skips a's forward stages. The CPU emulation follows the same order.

Result. GPU span 622 µs → 591 µs (median of 21 calls; the minimum improved by the same amount). The column pass on b alone also costs less than the old pass on both operands (16 µs against 22 µs).

4.8 Decompose: register blocking

Problem. `k_decompose` took 20.6 µs for half an operand, and the last call of it is on the critical path.

Diagnosis. Two guesses failed first: making L a compile-time constant (unrolled loops; 19.8 µs) and lazy reduction (19.4 µs), so neither latency nor arithmetic instruction count was the limit. Elimination showed the arithmetic phase was essentially the whole cost (19.4 µs full, 2.8 µs without the arithmetic, 19.1 µs without the stores). The SASS of `k_decompose<28>` showed, per residue, 58 global loads, 28 shared loads, 101 `IMAD` and 37 `IADD3`: **two table loads per term**, the two words $2^{32l} \bmod p$ and its Shoup companion, reloaded by every thread although a whole warp shares one prime.

Change.

- **Register blocking**: each thread computes 4 coefficients for the same prime (lanes take coefficients $\ell, \ell+32, \ell+64, \ell+96$), so each table entry loaded serves 4 terms, and every store is still 32 consecutive coefficients.
- **Lazy reduction**: each term is reduced only to $[0, 2p)$, terms are summed in 64 bits (below 2^{37} for $L \le 32$), and the sum $h \cdot 2^{32} + \ell$ is reduced once as $\ell \cdot 1 + h \cdot (2^{32} \bmod p)$, using entries 0 and 1 of the existing table.
- **Block size** 512 threads instead of 256, for enough warps per SM.

Result.

version	time (half operand)
original	20.6 µs
L templated	19.8 µs
+ lazy reduction	19.4 µs
+ 4 coefficients per thread, 256 threads	15.4 µs
+ 2 coefficients per thread, 512 threads	13.8 µs
+ 4 coefficients per thread, 512 threads	**12.2 µs**

4.9 NTT twiddles: one load instead of two

Diagnosis. By elimination, `k_ntt_rows_mul` (31.8 µs) split into about 14 µs of data movement, 8.4 µs of butterfly arithmetic and 9.4 µs of twiddle loads; `k_ntt_cols` (15.9 µs) into 10.1 µs, 2.5 µs and 3.3 µs.

Change. Each twiddle and its Shoup companion are stored side by side (`struct alignas(8) Twiddle`), so a butterfly fetches both with one 8-byte load.

Result. Only about 1 µs per kernel (31.8 µs → 30.8 µs, 15.9 µs → 15.3 µs). The twiddle cost is bytes, not instructions: a row-pass block needs about 2048 distinct twiddles per transform, so `k_ntt_rows_mul` reads about 30 MB of twiddles per call from L2. Kept because it is simpler and not slower.

5 Generalizing to Point #1

The original code accepted only $N \leq 2^{15}$, $L \leq 32$ and $K \leq 80$, and stopped in `fherma_init` at Point #1. Changes:

- Primes $p \equiv 1 \pmod{\max(2N, 2^{16})}$: the same set as before for $N \leq 2^{15}$, a valid set for larger N.
- N up to 2^{22} (beyond that, a row would not fit the 2048-element tile), L up to 64 ($W \leq 2048$), K up to 160; anything else is rejected with an explicit message.
- The CRT block's shared memory grows with K and L (about 52 kB at Point #1) and is now allocated dynamically, with the opt-in attribute set past 48 kB and a clear error if the device cannot provide it.

All 21 test points pass on the CPU and the GPU, including Point #1 with all-$(q-1)$ inputs, $W = 2048$ against the exact oracle and $N = 2^{17}$; the sanitizers are clean at $L = 55$ and $L = 64$. Register allocation shows no spills at $L = 55$.

The submission was also built and run inside FHERMA's challenge image `fherma/cupqc:0.6.0` (the digest shown on the leaderboard) from a clean copy of only the submitted files, with the platform's exact build command: no warnings, all points verified, and the same speed as the native build.

6 Optimizations for Point #1

At Point #1 the first working version spent 707 µs in kernels between the last input transfer and the first output transfer (against 95 µs at Point #2), with two kernels scaling far worse than the data, which grows by a factor of about 4 (K from 58 to 115, N doubled):

- `k_ntt_rows_mul`: 31 µs → 335 µs. The full twiddle tables are $16KN$ bytes = 121 MB, more than the L40S's 96 MB L2 (and, with the data, than the RTX PRO 6000's 128 MB), so they stream from DRAM.
- CRT: 12 µs → about 170 µs per chunk. Its shared memory had grown to 52 kB per block, which allows one 4-warp block per SM.

6.1 Twiddles generated from two small tables

Idea. A twiddle with index i is ψ^e with $e = \mathrm{bitrev}(i)$. Splitting $e = e_h 2^s + e_\ell$ with $s = \lceil \log_2 N/2 \rceil$,

$$\psi^e = \psi^{e_h 2^s} \cdot \psi^{e_\ell},$$

so two per-prime tables of $2^{\log_2 N - s}$ and 2^s entries suffice: 256 + 256 at Point #1, about 0.5 MB for all primes and both directions, against 121 MB. Stored in Montgomery form ($x \cdot 2^{32} \bmod p$), the twiddle is one Montgomery product of two table entries, $\mathrm{mont}(\psi^{e_h 2^s} 2^{32}, \psi^{e_\ell} 2^{32}) = \psi^e 2^{32}$, and applying it is a second one, $\mathrm{mont}(y, \psi^e 2^{32}) = y\psi^e$; no Shoup companion is needed.

Implementation. The butterflies take a *twiddle source* with one method, `mul(y, i)` $= y \cdot w_i \bmod p$: `TableTwiddles` reads the full Shoup table, `GenTwiddles` builds w_i as above. The row kernels are templated on the source. The column pass only ever uses indices below $2^{\lfloor \log_2 N/2 \rfloor}$, so it always reads a small head of the full table instead (235 kB at Point #1). The full tables are built only when the table path is chosen, which also saves 121 MB of device memory and part of the setup time at Point #1.

Result. Kernel times (Nsight Systems):

	Point #1		Point #2	
kernel	table	generated	table	generated
`k_ntt_rows_mul`	336	**132**	**31**	38
`k_ntt_rows_fwd`	120	**64**	**17**	20
`k_ntt_cols` (head table)	65	**55**	15	15

(times in µs). Generated twiddles win when the tables fall out of L2 and lose when they fit, so the choice is automatic: generated when $16KN$ exceeds 64 MB (Point #1), table otherwise (Point #2), overridable with `FHERMA_TWIDDLES`. The test matrix passes on both paths. GPU span at Point #1: 2.58 ms → 2.32 ms.

6.2 CRT split over limb ranges

Change. Instead of splitting each coefficient's sum over groups of primes, each of the block's 4 warps owns a range of output limbs: warp g accumulates the 64-bit columns of limbs $[g \cdot \mathit{span}, (g+1) \cdot \mathit{span})$, with $\mathit{span} = \lceil L/4 \rceil$, of $\sum_j y_j M_j$ over *all* primes, plus the rounding sum $\sum y_j/p_j$ over the primes $j \equiv g \pmod 4$. Lane i takes coefficient i of the block, so its y_j loads are 32 consecutive coefficients of one prime (coalesced, read straight from global memory) and its loads of M are uniform across the warp. Each partial is carried down to $\mathit{span} + 2$ words; a combine step per coefficient adds the 4 partials and $-vP \bmod q$, reduces modulo q, and writes the result over the same shared-memory row (odd row stride, so no bank conflicts). Shared memory no longer depends on K: about 10 kB at $L = 55$ instead of 52 kB.

Tuning. Kernel time per chunk:

variant	Point #2	Point #1
old: prime groups, Y staged in shared memory	11.9	about 170
limb ranges, 2 coefficients per thread	31.5	53.6
limb ranges, 1 coefficient per thread	15.9	50.0
+ prime loop unrolled by 4	**13.1**	**50.1**
+ unrolled by 8	18.3	50.3

(times in µs). Two coefficients per thread halve the number of blocks, and each iteration of the prime loop starts with a global load that has little arithmetic to hide behind; unrolling by 4 keeps several of those loads in flight. At Point #2 only the small first chunk is on the critical path, so the 1 µs lost there is negligible.

6.3 Host overhead at Point #1

The wall clock exceeded the GPU span by about 580 µs. The host timeline shows the input copies into pinned memory saturating at about 29 GB s^{-1} (1 ms for 28.8 MB) from 16 threads on, about the PCIe rate, so the first transfer waits for the whole first chunk (about 265 µs), and the last output chunk is copied out after the last transfer. Nothing tried helped measurably:

- more copy threads (24, 32, 48): copy time unchanged;
- seven chunk shapes, including a small first chunk of a and small last output chunks: all within 2.60 ms–2.81 ms, inside the noise;
- starting the output zero-fill only after the first input chunk, to free memory bandwidth for it: worse (2.69 ms → 3.08 ms), the fill then lands on the critical path.

On this VM the host side is bound by memory bandwidth; it may well be less so on the reference machine.

6.4 A container-only slowdown

Built and run inside FHERMA's image, Point #1 is 0.3–1 ms slower than natively (Point #2 only about 50 µs). The host timeline inside the container shows the helper's zero-fill of the 14.4 MB output vector taking about 3 ms instead of 1.3 ms. Page faults are the same in both. The image's glibc 2.39 zero-fills blocks above 2 kB with `rep stosb`, while the host's glibc 2.43 uses non-temporal stores above 256 kB; this VM's CPU model advertises neither ERMS nor FSRM, the features that make `rep stosb` fast. The slowdown is therefore probably specific to our VM; a fix that does not depend on the C library would be to zero-fill the next call's output vector in the background between calls, which moves work out of the timed window and is not done.

7 Getting closer to the transfer floor

Every output coefficient depends on every input coefficient, so the output cannot start before the whole input has arrived: the floor is the input transfer plus the output transfer. At Point #1 the wall clock (2.6 ms) exceeded that floor (1.8 ms on our Gen4 link) by about 400 µs–500 µs of host copies at the edges of the call and 370 µs of kernels between the last input byte and the first output byte.

7.1 Reading and writing the caller's memory through HMM

The driver here runs in HMM mode (heterogeneous memory management: GPU kernels can access ordinary pageable host memory). With the advice "preferred location: host, accessed by: GPU" (`cudaMemAdvise`), a kernel reads or writes a pageable buffer over PCIe without migrating its pages:

14.4 MB (one Point #1 operand)	time	rate
pinned DMA (reference)	593 µs	24.3 GB s^{-1}
kernel reads the pageable vector, no advice (pages migrate)	32 ms	0.4 GB s^{-1}
kernel reads it, advised to stay in host memory	684 µs	21.1 GB s^{-1}
prefetch (migrate) to the GPU, then read	6.3 ms	2.3 GB s^{-1}
kernel writes a pageable vector, advised to stay in host memory	732 µs	19.7 GB s^{-1}
previous output path (DMA, then one-thread copy)	2.8 ms	5.1 GB s^{-1}

So in the new default mode (`hmm`) the decompose kernel reads the limbs straight from the caller's input vectors, and the CRT writes the result straight into the output vector (after the helper's zero-fill): there are no host staging copies at all. The output stays in host memory, so nothing is deferred past the end of the call. Where the device cannot access pageable memory the code falls back to the staged path.

Two consequences followed. First, in this mode b's decompose kernels *are* b's transfer, so a's forward NTT moved to its own stream, with lower priority than the decompose stream (otherwise b's first decompose started 46 µs late, behind the NTT's blocks). Second, the decompose kernel now loads 16 bytes at a time for full blocks, which raised its read rate from 18.4 to about 20 GB s^{-1}.

7.2 Chunking in HMM mode

With no host copy to overlap, every chunk boundary only leaves the end of a kernel underusing the GPU, and here the kernels are the transfers. GPU span at Point #1:

input chunks / output chunks	GPU span
2 / 1:2:1 (staged defaults)	2404 µs
8 / 1:2:1	2705 µs
1 / 1:2:1	2323 µs
2 / 1	2304 µs
1 / 1	**2242 µs**

One chunk each way is the default in `hmm` mode.

Result. Interleaved medians, native: Point #1 2616 µs → about 2300 µs; Point #2 684 µs → about 610 µs. Host overhead fell from about 400 µs–500 µs to about 60 µs at Point #1. All 21 test points pass in every copy mode; sanitizers clean.

7.3 Tried and not adopted

- **A pinned memory pool behind a replaced global `operator new`**, so that the harness's own input and output vectors would be pinned and DMA-able at the full 24.5 GB s^{-1}. Built only as a scratch experiment: it deadlocked inside the CUDA runtime, even with a re-entrancy guard and with the pool enabled only after initialization. It would also change how the harness allocates its buffers, so it would need FHERMA's approval. Expected gain over `hmm`: the DMA-versus-HMM rate difference, about 250 µs–300 µs at Point #1.

- **A single-pass NTT per prime using thread-block clusters** (distributed shared memory), which would cut the memory passes after the last input byte from three to one. Clusters need `sm_90` or newer; the L40S cannot run them, so this was not attempted.
- **A measurement on FHERMA's hardware**: it requires pushing the code to a repository and registering the commit from the participant's account.

7.4 The zero-fill inside the challenge image

Inside `fherma/cupqc:0.6.0` on this VM, every mode waits on the output vector's zero-fill, which takes 1.5–2.9 ms there depending on load, against about 1.3 ms natively (the image's glibc 2.39 `memset` against the host's 2.43). In `hmm` output mode the CRT has to wait for it, so measured in the image (three rounds) the modes overlap: staged 3.1 ms–3.7 ms, `hmm` 2.5–3.6 ms. This is probably specific to our VM's old CPU model, but it is unverified for FHERMA's machine. The only complete fix found, preparing the next call's zeroed output vector between calls, moves work out of the timed window and is not done.

7.5 Further attempts

- **Huge pages for the HMM buffers.** With the advice calls outside the timing (as the code caches them), a streaming kernel already reaches 23.9 GB s^{-1} reading and 25 GB s^{-1} writing pageable memory with 4 kB pages; collapsing the range to 2 MB pages changed nothing. The shortfall against DMA is in our kernels, which stop reading or writing while they compute.
- **A separate streaming copy kernel** (`stream` mode) moves data between the caller's memory and device memory at full speed, but it starves the decompose of SMs (902 µs instead of 72 µs for the first chunk): slower overall (GPU span 2452 µs against 2245 µs). Kept as an option.
- **Two decompose streams.** Alternating decompose chunks between two equal-priority streams lets each chunk start while the previous one finishes: two chunks now cost the same as one, eight cost 2372 µs instead of 2705 µs.
- **Column-group streaming** of b's column pass and of the inverse column pass with the CRT was not built: it needs about eight groups per operand, and eight decompose chunks still cost about 120 µs more than one, against about 50 µs of hidden work.
- **A software-pipelined decompose** (persistent blocks, two shared-memory tiles, `cp.async` loads for the next tile while the current one is computed): correct, but slower (895 µs per Point #1 operand against 667 µs): the two tiles take 56 kB, so one block per SM, and word-by-word asynchronous copies cost more instructions than they save.
- **Choosing the copy mode at setup.** Which mode wins depends on the machine, so `fherma_init` times the candidates on dummy operands (seven runs each, evicting the caches between runs to mimic the harness's file I/O) and keeps the default unless another mode is at least 5 % faster.

8 Where the time goes now

8.1 Point #2

Wall clock about 700 µs median per call; GPU span about 590 µs. One call:

time (µs)	activity
0–326	four 1.84 MB host-to-device transfers, back to back; a's decompose and forward NTT hidden under them
326–434	about 95 µs of kernels: last decompose, column pass on b, `k_ntt_rows_mul`, inverse column pass, first CRT chunk
434–582	device-to-host transfers of 0.92 MB, 1.84 MB and 0.92 MB, back to back, with the remaining CRT chunks underneath

About 470 µs of the span is PCIe transfer at full speed; what remains is about 95 µs of kernels and 25 µs–100 µs of host work at the edges.

8.2 Point #1

Wall clock about 2.3 ms median natively (first version: 3.09 ms); GPU span 2.24 ms. One call, in `hmm` mode:

time (µs)	activity
0–1440	input: a's and b's decompose kernels reading the caller's vectors over PCIe (about 20 GB s^{-1}); a's forward NTT on its own stream underneath
1440–1684	244 µs of kernels: column pass on b, `k_ntt_rows_mul`, inverse column pass (about four memory passes over the residues)
1684–2340	output: the CRT writing the result straight into the output vector over PCIe

Host overhead outside the GPU span is about 60 µs. What remains above the floor is the 244 µs of kernels, plus the gap between HMM access (20–22 GB s^{-1}) and DMA (24.5 GB s^{-1}).

9 What did not help

attempt	outcome
CRT: staging Y in shared memory alone	308 µs → 232 µs; the big gain came from prime groups
More chunks after the copy pool	slower: fixed cost per parallel copy
Uneven input chunks	neutral or worse (GPU span 625 µs/647 µs vs. 591 µs–610 µs)
Zero-copy (`mapped`) transfers	about 2 %, within noise; kept opt-in
Spin loop without `pause`	no difference
Decompose: templated L; lazy reduction alone	20.6 µs → 19.8 µs → 19.4 µs; the limit was table loads
Interleaved twiddle table	about 1 µs per NTT kernel; the cost is bytes
Generated twiddles at Point #2	slower (`k_ntt_rows_mul` 31 → 38 µs); the tables fit in L2 there, hence the automatic rule
CRT: 2 coefficients per thread; unroll by 8	fewer blocks / more registers: slower
Point #1 host side: more threads, chunk shapes, later zero-fill	no gain; the later zero-fill was worse

10 Summary at Point #2

stage	median per call
first GPU run (GPU shared, best setting)	3.07 ms
same day, old code, idle GPU (best setting)	2.14 ms
after the copy pool and shallow pipeline	about 880 µs
after spinning workers	about 760 µs
after helper zero-fill and 1:2:1 output chunks	about 700 µs
inside FHERMA's challenge image	703 µs

The NTT overlap and the decompose and twiddle changes shortened the GPU span by about 40 µs more, inside the wall-clock noise.

A Runtime settings

variable	default	meaning
`FHERMA_IN`, `FHERMA_OUT`	`hmm`	`hmm`, `staged`, `register`, `mapped`
`FHERMA_CHUNKS`	1 (`hmm`), 2	input chunks (equal)
`FHERMA_OUT_CHUNKS`	—	output chunks (equal)
`FHERMA_A/B/C_CHUNKS`	C: `1` (`hmm`), `1,2,1`	chunk weights per operand
`FHERMA_THREADS`	$\min(16, \text{cores}/2)$	host copy threads
`FHERMA_SPIN_MS`	20	spin after a call before sleeping
`FHERMA_TWIDDLES`	`auto`	`table`, `gen` or `auto` (by table size)
`FHERMA_PROFILE`	off	per-phase GPU times and host timeline

B Reproducing the measurements

```
./build.sh                                    # all binaries (see HANDOVER.md)
./run_tests.sh && ./run_tests.sh build/solution
python3 make_big.py /tmp/p2 32768 868 21     # Point #2
python3 make_big.py /tmp/p1 65536 1747 21    # Point #1
build/solution /tmp/p2 && python3 oracle.py verify /tmp/p2
nsys profile -t cuda --sample=none --cpuctxsw=none -o prof build/solution /tmp/p2
./docker_check.sh 0                           # build and run in fherma/cupqc:0.6.0
```